4/29/26

Ahmed

Neural Network HW

10) (a) Training MSE $1.027 \cdot 10^{-7}$

(b) ~~More training~~ More training improves the approximation. Does well on training points w/ only 20 points ($1.027 \cdot 10^{-7}$) but generalizes poorly to the test set (8.786), while w/ 200 points trained got $2.229 \cdot 10^{-4}$ test MSE. More info to learn network shape.

Overtraining causes degradation. train MSE went to 0, or $1.013 \cdot 10^{-23}$ but got test score of 9.963

~~(c) the linear network performs worse~~

Case	Train MSE	Test MSE
20 trained	$1.027 \cdot 10^{-7}$	8.786
200 trained	—	$2.29 \cdot 10^{-4}$
20 overtrained	$1.013 \cdot 10^{-23}$	9.96

(C) the performance worsened w/ linear activations. Fully linear collapsed to simple affine map regardless of neurons. However, sigmoid's non-linear activation let training MSE drop to 0

	Train MSE	test MSE
Linear	6.198	5.545
Sigmoid	$1.97 \cdot 10^{-16}$	4.461

Neural Networks HW

(d)

the two layers sigmoid gave best MSE. 2 layer 200 points ($2.181 \cdot 10^{-5}$) moderately better than simple 200 points ($2.24 \cdot 10^{-4}$), training times was similar. LM handled extra weights. Worse overtraing w/ 2 layers time more to memorize. 2 & 1 layer linear bc Linear + Linear composition is still linear.

Case	Train MSE	Test MSE	Time (s)
20pts 2-layer sigmoid	$4.827 \cdot 10^{-12}$	1.614	0.52
200pts 2-layer sigmoid	$9.949 \cdot 10^{-7}$	2.181	0.84
Overtrained	$1.074 \cdot 10^{-27}$	7.535	6.25
20pts 2-layer linear	6.198	5.345	0.45

Chapter 6

System: $y(k) = y(k-1)(1 + y(k-1)^2) + u(k-1)^3$

$f(y(k-1)) = \frac{y(k-1)}{1 + y(k-1)^3}$

Method	test MSE	time
trainim	0.74	$6.33 \cdot 10^{-1}$
trainyd	4.08	1.432

— 2000 training samples w/ random $u \sim U(-1, 1)$ with [10 10] hidden layers using tansig/purelin functions with a learning rate of .01

trainlm converged 5x faster and half the test MSE of traingd. Since LM approximates the Hessian, it takes well-scaled steps and quickly converges. On the other hand, traingd only uses the first order gradient information to converge and must use a small learning rate, such that many iterations are needed to converge. Thereby, trainlm is more ideal for these networks.

5)

I'm assuming HW #2 data 4-2???
No Nguyen is likely standard single tank single tank / mamdani rule base

$$\text{Tank: } \frac{dh}{dt} = \frac{q_{in} - C\sqrt{h}}{A}, \quad C=.6,\ A=1,\ h_{ref}=2m,\ T_s=.1s$$

Controller inputs = $h_{ref} - h$ & de/dt
outputs = $\int \Delta v$, saturated to $[0,5]$

q_{in}

Fuzzy Baseline: Mamdani controller w/ 5 triangular MFs per I/O (NB/NS/ZE/PS/PB) + 5x5 rule table
Fuzzy IAE = 4.125

(a) NN training params trained on fuzzy controller $(e, de) \to dv$ data w/ plant parameters

NN HW

Feed forward: [10 10] layers, trainlm, tansig/tansig/lineem, $1e^{-6}$ goal for 500 epochs

RBF: newrb, $1e^{-4}$ goal, w/ spread of 1 and 60 neurons at most

model	training params
FF	$6.392 \cdot 10^{-7}$
RBF	$6.592 \cdot 10^{-5}$

(b)

Controller	ISE
Fuzzy	4.125
Feed forward NN	4.107
RBF NN	4.066

all 3 controllers stabilized h_ref w/ similar ISE values expected since trained on fuzzy controller I-O mapping so basically just approximating it. RBF had the lowest ISE, likely since guassian RBFs are structurally similar to Guassian fuzzy membership functions in the original controller. Basically the output weights were kind of like rules so it fit the problem pretty well. However, since the NNs have smooth sigmoidal differentiables, they aren't interpretable by a human like with a fuzzy signal.